Volume 02

Core Platform Architecture

Version: 1.0 (Draft)

Status: Design

Language: English

Chapter 1 – Architectural Vision

1.1 Purpose

The purpose of the Core Platform is to provide a unified, modular and scalable foundation for every capability within Project Sentinel.

Every module shall operate as part of one coherent platform rather than as an independent application.

1.2 Design Goals

The Core Platform shall:

- provide a unified architecture
- support unlimited modules
- remain easy to use
- scale from personal computers to enterprise environments
- support local deployment
- support cloud deployment
- support hybrid deployment
- remain technology independent where practical
- minimize technical debt
- enable long-term maintainability

1.3 Fundamental Principles

The Core Platform is based on the following engineering principles:

- Object-Oriented Domain Model
- Event-Driven Architecture
- API-First Design
- Plugin-Based Extensibility
- Security by Design
- Privacy by Design
- Evidence-Based Analysis
- Explainable Recommendations
- Human-Centered Experience

Chapter 2 – Platform Layers

The platform is divided into logical layers.

Chapter 3 – Platform Characteristics

The platform shall have the following characteristics:

Modular

Each capability shall be implemented as an independent module.

Replaceable

Modules shall be replaceable without affecting the entire platform.

Observable

Every action shall produce observable events.

Auditable

Every decision shall be traceable.

Explainable

Every recommendation shall include supporting evidence.

Extensible

New capabilities shall be installable without redesigning the platform.

Chapter 4 – Core Components

The platform consists of the following core engines.

Core Object Engine

Maintains every object.

Knowledge Engine

Maintains relationships.

Evidence Engine

Maintains evidence.

Workflow Engine

Coordinates processes.

Event Engine

Processes platform events.

Notification Engine

Delivers notifications.

Reasoning Engine

Produces recommendations.

Policy Engine

Applies organizational policies.

Health Engine

Calculates health metrics.

Search Engine

Indexes platform knowledge.

Visual Engine

Generates interactive visualizations.

Chapter 5 – Platform Modules

The first planned modules are:

- Asset Management
- Discovery
- ASM
- Vulnerability Assessment
- Compliance
- Identity Security
- Certificate Management
- Cloud Security
- Container Security
- Kubernetes Security
- Mobile Security
- Email Security
- Web Security
- API Security
- Threat Intelligence
- IOC Management
- Case Management
- SOC
- SIEM
- Reporting
- Automation
- AI Assistant
- Digital Twin
- Knowledge Graph
- Visual Analytics
- Risk Management
- Secrets Management
- Configuration Assessment
- Backup Assessment
- Supply Chain Security
- SBOM Analysis
- Plugin Marketplace

Chapter 6 – Platform Rules

Every module shall:

- expose a documented API
- support role-based authorization
- produce audit events
- generate structured logs
- support localization
- expose health information
- support automated testing
- support versioning
- follow the platform UI guidelines

Chapter 7 – Scalability

Project Sentinel shall support deployment as:

- Single-user desktop installation
- Small business server
- Enterprise deployment
- Government deployment
- Air-gapped environment
- Cloud-native installation
- Hybrid installation

without redesigning the architecture.

Chapter 8 – Technology Choices (Initial)

These are **current design targets**, not irreversible decisions.

```
| Component | Initial Choice | Reason
```

```
| Language | Python | Fast development, large ecosystem
```

New Design Principle – "Core vs Integration"

This is one of the most important architectural decisions.

We separate everything into two categories:

Core

We own it completely.

Examples:

- Object Engine
- Knowledge Engine
- Reasoning Engine
- Workflow Engine
- Digital Twin
- Mission Control

- Visual Analytics
- Evidence Engine

These are unique to Project Sentinel.

Integrations

We integrate with well-established tools and standards where they provide value.

Examples include:

- Network scanning libraries.
- Cryptographic libraries.
- Operating system APIs.
- Standard vulnerability data formats.
- Identity protocols.
- Cloud provider APIs.

We will avoid rewriting mature, security-critical building blocks unless there is a compelling engineering reason.

A New Long-Term Goal

I would like to introduce one final guiding principle for the Blueprint:

Every feature must answer one question:

> **"Does this help the user make a better security decision?"**

If the answer is **no**, we should question whether the feature belongs in Project Sentinel at all.

That principle will keep the platform focused, usable, and valuable as it grows. I also suggest that from the next volume onward we start assigning formal requirement IDs (PS-REQ), module IDs (PS-MOD), architecture decision records (PS-ADR), and object IDs (PS-OBJ), so every part of the Blueprint becomes traceable from design through implementation and testing. This level of discipline is what will make Project Sentinel a professional engineering project rather than just an ambitious software idea.

=====[USER]=====

Ok fortsett

=====[ASSISTANT]=====

Flott. Nå går vi inn i det jeg mener blir den **viktigste tekniske delen av hele Project Sentinel**.

Alt vi bygger senere (Discovery, Mobile Security, SOC, AI, Compliance osv.) kommer til å bygge på dette.

Fra nå av skal vi tenke som programvarearkitekter som bygger et operativsystem.

PROJECT SENTINEL BLUEPRINT

Volume 03